



Chat overlay

Embed a running chat with a typed message protocol.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-chat-overlay



When this library is useful

Embed a running chat with a typed message protocol.

01

Embed a full experience

Mount the chat iframe in a host-owned region.

02

Control conversations

Set input, send requests and subscribe to events.

03

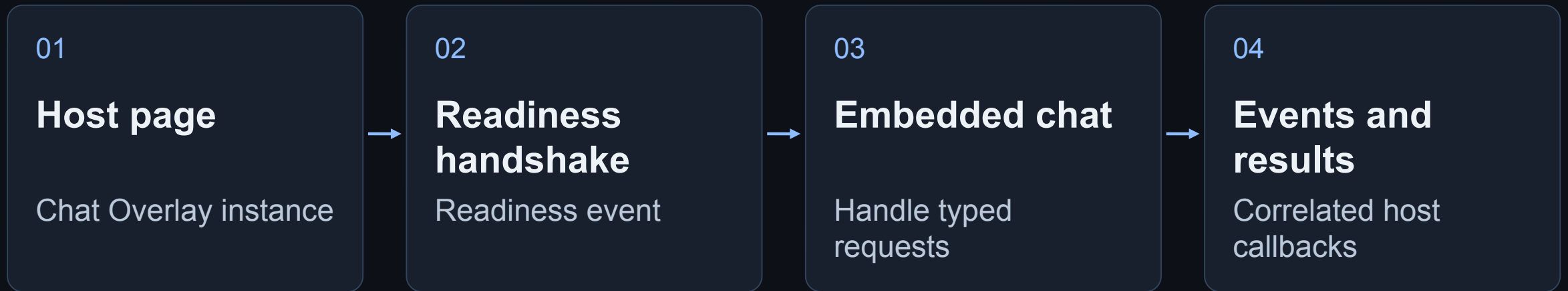
Offer floating widgets

Use ChatOverlayManager for toggle/fullscreen chrome.



Follow the ownership boundary

Inputs and effects stay explicit.



Library: Iframe lifecycle, protocol handshake, request correlation and optional widget chrome.



The public contract to start with

ChatOverlay · selected surface; full API index is in the bundle.

Input or operation	Contract
<code>new ChatOverlay(root, options)</code>	Mount and configure one iframe
<code>ready()</code>	Promise for the readiness handshake
<code>setInputContent(text)</code>	Request to the embedded chat
<code>subscribe / destroy</code>	Event lifecycle and cleanup



A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
const overlay = new ChatOverlay('#demo-chat', {  
  domain: 'https://chat.example.com',  
  theme: 'dark',  
});  
await overlay.ready();  
await overlay.setInputContent('Explain the architecture');  
// On host teardown:  
overlay.destroy();
```

Full example: [examples/chat-overlay.ts](#) · uses [@epam/ai-dial-chat-overlay](#)



Wire a realistic host workflow

Keep chat overlay focused on its public responsibility.

01

Mount the iframe

Create ChatOverlay in an existing host-owned element.

02

Wait for readiness

Await ready() before sending the initial input content.

03

Match host lifetime

Subscribe to events; unsubscribe and destroy on teardown.



Subscribe and clean up

Match event ownership to the host page lifecycle.

```
const unsubscribe = overlay.subscribe(  
  OverlayEventType.GptStartGenerating,  
  () => console.info('Demo generation started'),  
);  
  
// On host teardown:  
unsubscribe();  
overlay.destroy();
```

Focused source-backed example · see [examples/](#) and [content/api-index.md](#).



Available customization

01

Any browser framework

The overlay implementation uses DOM/TypeScript rather than React.

02

Lifecycle contract

Await ready, subscribe deliberately and destroy on unmount.

03

Two integration levels

Direct overlay for a container; manager for floating widgets.



Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

Evidence	Where to look
Real consumer	apps/chat-overlay-sandbox/src/cases/ConversationListCase/ ConversationListCase.tsx
Test evidence	throws a descriptive error when the root selector matches nothing
Test evidence	throws a descriptive error for an unknown overlayId



Constraints that affect integration

01

Embedding policy

The server must allow the host origin in frame-ancestors.

02

Auth constraints

Provider iframe behavior varies; configure supported login UI modes.

03

No server replacement

The overlay still needs a running chat and backend.